

LAB ACTIVITY: IOT WEBSOCKET INTEGRATION DESIGN

Designing a Real-Time IoT Monitoring System Using WebSocket

Student Name: Bradley Sheen Sale

Course/Year: BS Information Technology - 3rd Year

Date: May 17, 2026

PART A: SYSTEM ARCHITECTURE DESIGN

The proposed Smart Agriculture Monitoring System utilizes a **Layered Bi-Directional Architecture**. This design ensures that raw telemetry from the field reaches the end-user with sub-second latency, bypassing the limitations of traditional HTTP request-response cycles.

System Components:

- 1. Perception Layer (IoT Device / Sensor):** ESP32 microcontrollers equipped with DHT22 sensors (for temperature and humidity) and drone-mounted OV2640 cameras (for crop visual health). These act as the primary WebSocket clients transmitting raw JSON telemetry.
- 2. Edge Layer (Optional Edge Device):** A local on-site gateway (e.g., Raspberry Pi) that aggregates data from multiple sensor nodes. It performs data sanitization and runs a lightweight heuristic check to trigger immediate local actuators (like valves) before sending data to the cloud.
- 3. Transport Layer (WebSocket Server):** A central backend powered by **Django Channels** and Redis. It acts as the connection broker, maintaining persistent TCP tunnels with both the field hardware (ingress) and the user dashboards (egress).
- 4. Intelligence Layer (AI / Data Model):** A Python-based predictive inference engine (using PyTorch or TensorFlow) integrated into the backend. It continuously analyzes the incoming WebSocket data stream to classify plant health.

5. **Application Layer (Web Client / Dashboard):** A ReactJS-based frontend interface. It maintains a persistent WebSocket connection to the server, instantly updating visual gauges, historical charts, and alert banners without requiring a page refresh.

[SYSTEM ARCHITECTURE DIAGRAM]

(IoT Sensors / Drones) → (Edge Gateway) → [ws://] → (Django WebSocket Server)
(Django Server) → [Inference] → (AI Python Engine)
(Django Server) → [Broadcast] → (ReactJS Web Dashboard)

Guide Questions:

1. **Which components communicate using WebSocket?** The IoT/Edge Devices communicate upstream to the WebSocket Server, and the WebSocket Server pushes that data downstream to the Web Client (Dashboard).
2. **Where does data processing or model inference occur?** Preliminary processing and immediate threshold alerts occur at the Edge Device. Deep trend analysis and complex model inference occur at the Server-side Intelligence Layer where there is higher computing power.
3. **Why is WebSocket preferred over REST for live monitoring?** REST is a stateless, unidirectional protocol that relies on "HTTP Polling"—wasting massive amounts of bandwidth and increasing latency. WebSocket is a stateful, full-duplex protocol. Once the handshake is complete, the server can "push" data to the dashboard the exact millisecond a sensor reading changes, enabling true real-time monitoring.

PART B: DATA FLOW DESCRIPTION

1. **Sensor Data Generation:** The field sensors capture ambient temperature, humidity, and visual leaf data, formatting it into a lightweight JSON payload (e.g., {"temp": 32.5, "hum": 45, "img_status": "yellowing"}).
2. **WebSocket Connection Establishment:** When the devices power on, both the Edge Gateway and the Web Dashboard send an HTTP Upgrade request to the

Django server. The server responds with a 101 Switching Protocols status, establishing a permanent, bi-directional TCP tunnel.

3. **Data Transmission:** The IoT Edge device streams the JSON data frames continuously through the open socket to the Django Channels consumer without the overhead of HTTP headers.
4. **Model Analysis:** The server routes the incoming payload to the Python AI Model. The model evaluates the multimodal data, correlating high temperature and visual yellowing to classify the crop health status.
5. **Real-Time Update:** The server instantly broadcasts a compiled JSON packet containing both the raw telemetry and the AI classification to all subscribed Dashboard clients, updating their UI instantly.

PART C: MODEL INTEGRATION

1. **What type of data will the model receive?** The model will receive **Multimodal Data**. This includes continuous time-series numerical floats (temperature and humidity levels) and extracted image feature arrays (pixel data from drone visuals).
2. **How will WebSocket deliver model predictions to the client?** Instead of the client requesting the prediction, the WebSocket server wraps the model's output into a standardized JSON text frame and executes a "server-side push" over the existing open connection directly to the browser's JavaScript WebSocket listener.
3. **What happens when abnormal data is detected?** When the model detects a "Critical" anomaly, the WebSocket server executes a dual-action response. It instantly pushes a visual and auditory alert to the Web Dashboard, and simultaneously transmits a command frame back down the WebSocket tunnel to the Edge Device to autonomously trigger an actuator (e.g., opening an irrigation valve).

PART D: REFLECTION

WebSocket technology significantly improves the scalability and performance of IoT systems by eliminating the massive network overhead inherent in traditional HTTP polling. In a polling-based architecture, a client must repeatedly establish connections and transmit heavy HTTP headers just to check for updates, which clogs network bandwidth and exponentially increases server CPU load even when sensor data remains unchanged. WebSockets resolve this by maintaining a

single, long-lived TCP connection that remains idle until an actual event occurs. This event-driven, "push" architecture drastically reduces latency to the millisecond scale, ensuring critical agricultural anomalies are reported instantly. Furthermore, because data is transmitted in ultra-lightweight frames, a single server can efficiently manage thousands of concurrent sensor nodes and dashboards. Ultimately, transitioning from REST to WebSockets transforms a passive data-logging system into an active, highly responsive, real-time ecosystem.